

# Week 15 – Efficiency & Optimization

## ■ Learning Objectives

- Interpret time limits and estimate complexity.
- Refactor nested loops; leverage sets/dicts and sorting.
- Use early exits, pruning, and precomputation.

## ■ Complexity Basics (No Heavy Math)

- $n$  up to  $\sim 100 \Rightarrow O(n^2)$  okay;  $n$  up to  $\sim 10^5 \Rightarrow$  need  $O(n \log n)$  or  $O(n)$ .
- Constant factors matter: avoid repeated work inside loops.
- Replace 'in list'  $O(n)$  with 'in set'  $O(1)$  average.

### *Example – Speeding Up Lookups*

```
# Slow:  $O(n^2)$  membership checks
arr = list(range(10000))
targets = [3, 7000, 9999]
hits = 0
for t in targets:
    if t in arr:
        hits += 1

# Fast:  $O(n + q)$  using a set
s = set(arr)
hits2 = sum(1 for t in targets if t in s)
print(hits, hits2)
```

## ■ Practice Problems

### ***Practice 1 – Pair Sum in $O(n)$***

```
arr = [2, 7, 11, 15]
T = 9
seen = set()
ans = None
for x in arr:
    need = T - x
    if need in seen:
        ans = (need, x); break
    seen.add(x)
print(ans)
```

### ***Practice 2 – Prefix Sums***

```
nums = [1, 2, 3, 4, 5]
pref = [0]
for x in nums:
    pref.append(pref[-1] + x)

# range sum [l,r] inclusive
l, r = 1, 3
print(pref[r+1] - pref[l])
```

## ■ Optimization Tips

- Read all input first (fast I/O) when needed.
- Move invariant computations outside loops.
- Short-circuit with break/continue when possible.

## ■ Homework Challenge

Given N integers and Q queries (l,r), output range sums quickly using prefix sums.

```
nums = [3,1,4,1,5,9]
queries = [(0,2),(2,5)]
pref = [0]
for x in nums: pref.append(pref[-1]+x)
for l,r in queries:
    print(pref[r+1]-pref[l])
```